Universidad Nacional de Asunción
Facultad Politécnica

Ingeniería en Informática



Verificación de consenso e inclusión de transacciones en Ethereum con conocimiento cero aplicado a la transferencia de activos digitales

Javier Rafael López Cáceres
Alejandro Fabián Marín Insfran

San Lorenzo - Paraguay
Septiembre - 2026

Universidad Nacional de Asunción
Facultad Politécnica

Ingeniería en Informática



Verificación de consenso e inclusión de transacciones en Ethereum con conocimiento cero aplicado a la transferencia de activos digitales

Javier Rafael López Cáceres
Alejandro Fabián Marín Insfran

Asesor:
PhD Marcos Villagra

Proyecto de Trabajo de Grado presentado en conformidad a los requisitos para obtener el grado de Ingeniería en Informática

San Lorenzo - Paraguay
Septiembre - 2026

Dedico djkfsadkfamas dfasdk,
alsdalsd.

AGRADECIMIENTOS

Verificación de consenso e inclusión de transacciones en Ethereum con conocimiento cero aplicado a la transferencia de activos digitales

Autor: Javier López - Alejandro Marín

Asesor: PhD Marcos Villagra

RESUMEN

Los puentes entre cadenas o popularmente conocidos como *cross-chain bridges*, permiten transferir activos entre distintas *blockchains* y constituyen una pieza central de la interoperabilidad. La mayoría se apoya en el patrón *lock-and-mint* operado por un *relayer* de confianza, un actor que tras observar el bloqueo de fondos en la cadena de origen acuña el equivalente en la cadena de destino. Como toda la seguridad recae en ese operador, que podría acuñar de forma fraudulenta, los puentes se han vuelto uno de los blancos más explotados del ecosistema. En este trabajo se diseña e implementa un puente de confianza reducida (*trust-minimized*) que sustituye esa confianza por verificación criptográfica. El contrato de destino solo acuña cuando valida *on-chain* una prueba de conocimiento cero (*zero-knowledge*) que demuestra tanto la inclusión real de la transacción de bloqueo en un bloque de la cadena de origen a través de una prueba de inclusión en el árbol de transacciones, como la coincidencia del destinatario y el monto de la transferencia con esa transacción, de modo que no puedan adulterarse. Los circuitos se desarrollaron en Noir y sus verificadores se desplegaron *on-chain* en Solidity. Un *relayer off-chain* construye la prueba de cada operación y la remite al contrato de destino, donde deja de ser un acuñador de confianza y pasa a ser un probador sin permisos, ya que cualquiera con una prueba válida puede liberar los fondos y nadie puede acuñar sin ella. De manera complementaria se diseña la validación de la firma del comité de sincronización (*sync committee*) de Ethereum como raíz de confianza sobre la finalidad del bloque. La solución se validó sobre redes de prueba reales, con Ethereum Sepolia como cadena de origen y Optimism Sepolia como destino. Adicionalmente incluye una interfaz web y un modo comparativo que evidencia de forma experimental cómo el esquema clásico admite la acuñación fraudulenta mientras que el esquema con verificación de conocimiento cero la rechaza.

Palabras Clave: puentes entre cadenas (*cross-chain bridges*); pruebas de conocimiento cero (*zk-SNARKs*); Noir; UltraHonk; verificación *on-chain*; interoperabilidad *blockchain*; prueba de inclusión (árbol de Merkle-Patricia); confianza reducida (*trust-minimization*); Ethereum.

Title

Author: full name

Advisor: full name

ABSTRACT

This is the abstract

Keywords:

ÍNDICE

Página

1. INTRODUCCIÓN **1**

1.1. Contexto y antecedentes . . . 1
1.2. Planteamiento del problema . . . 2
1.3. Justificación . . . 2
1.4. Objetivos . . . 3
1.4.1. Objetivo general . . . 3
1.4.2. Objetivos específicos . . . 3
1.5. Hipótesis . . . 4
1.6. Delimitación . . . 4

2. BLOCKCHAIN **5**

2.1. Arquitectura de blockchain . . . 5
2.1.1. Estructura de bloques y encadenamiento por hash . . . 5
2.1.2. Árboles de Merkle y pruebas de inclusión . . . 6
2.1.3. Mecanismos de consenso . . . 6
2.1.4. Roles de nodos y topología de red . . . 7
2.2. Ethereum (Estructura de bloques) . . . 8
2.3. Aztec (Estructura de bloques) . . . 8
2.4. Puentes . . . 8

3. CONOCIMIENTO CERO **9**

3.1. Fundamentos . . . 9
3.1.1. Concepto de conocimiento cero . . . 9
3.1.2. Propiedades fundamentales . . . 11
3.1.3. Pruebas interactivas y no interactivas . . . 12
3.1.4. De una computación a una prueba . . . 12
3.1.5. SNARKs y STARKs . . . 15
3.2. Circom . . . 17
3.3. Noir . . . 17
3.4. SP1 . . . 17
3.5. Otros (Risc0, Succinct, Aligned, Cairo, Halo2, Plonky3) . . . 17

4. ZKBRIDGE **18**

4.1. Implementación de un puente Lock-and-Mint . . . 18
4.2. zkBridge de Berkeley RDI . . . 18
4.3. Diseño e Implementación en Noir . . . 18
4.4. Benchmarks . . . 18

5. CONCLUSIÓN **19**

REFERENCIAS **20**

APÉNDICE **24**

A.1. A 24

LISTA DE FIGURAS

Página

3.1. Etapas de generación y verificación de una prueba. 14

LISTA DE TABLAS

Página

3.1. Diferencias conceptuales entre SNARKs y STARKs. 16

LISTA DE SÍMBOLOS

H	Función de hash criptográfica.
$\|\|$	Concatenación de cadenas de bits.
$\mathcal{P}$	Probador (*prover*) de un sistema de prueba.
$\mathcal{P}^*$	Probador potencialmente deshonesto.
$\mathcal{V}$	Verificador (*verifier*) de un sistema de prueba.
x	Enunciado (*statement*); entrada pública.
w	Testigo (*witness*); entrada privada del probador.
π	Prueba (*proof*) generada por el probador.
$\mathcal{R}$	Relación que define los pares válidos de enunciado y testigo.
$\mathbb{F}_p$	Cuerpo finito de p elementos, con p primo.

CAPÍTULO 1
INTRODUCCIÓN

1.1. Contexto y antecedentes

Ethereum se ha consolidado como la plataforma de referencia para el despliegue de contratos inteligentes y constituye la base sobre la que se sostiene una porción significativa del ecosistema de finanzas descentralizadas [1]. Este crecimiento no se produjo de forma aislada, sino que estuvo acompañado por la proliferación de redes de *Layer 2* y cadenas alternativas que replican o extienden su modelo de ejecución, cada una optimizada según distintos criterios de costo, rendimiento o descentralización. La fragmentación resultante de este ecosistema plantea la necesidad de que los activos digitales y los datos puedan trasladarse entre las distintas cadenas que lo componen, requisito que se conoce como interoperabilidad. Los puentes entre cadenas, o *cross-chain bridges*, constituyen la infraestructura que hace posible dicho traslado, de modo que la seguridad de estos mecanismos determina, en gran medida, la seguridad de los fondos que a través de ellos se transfieren.

Paralelamente al desarrollo de estas redes, las pruebas de conocimiento cero (*Zero-Knowledge Proofs*) evolucionaron desde ser un objeto de estudio predominantemente teórico hasta convertirse en una herramienta de ingeniería aplicable a problemas concretos. Esta tecnología permite comprimir la verificación de un cómputo arbitrariamente extenso en una prueba sucinta, cuya validación puede realizarse en una fracción del tiempo y del costo que exigiría reejecutar dicho cómputo. Aplicada al contexto de Ethereum, esta propiedad habilita la verificación liviana, desde otra cadena, tanto del consenso alcanzado por los validadores —a través de la firma emitida por el comité de sincronización que certifica la finalidad de un bloque— como de la inclusión efectiva de una transacción particular dentro de ese bloque, lo cual se logra mediante una prueba sobre su árbol de Merkle-Patricia. La combinación de ambas verificaciones, comprimidas en una única prueba de conocimiento cero, constituye el fundamento técnico sobre el que resulta

posible construir un puente entre cadenas cuya seguridad no dependa de la honestidad de un intermediario, sino de una demostración matemáticamente verificable. En el presente trabajo, este fundamento se materializa mediante el diseño e implementación de un puente del tipo *lock-and-mint*, en el cual el bloqueo de un activo en la cadena de origen habilita su acuñación equivalente en la cadena de destino únicamente cuando dicha prueba ha sido validada *on-chain*.

1.2. Planteamiento del problema

La verificación directa del consenso de Ethereum y de la inclusión de una transacción, sin comprimir dicho cómputo mediante una prueba de conocimiento cero, resulta impracticable cuando se pretende realizar fuera de la propia cadena. Reproducir la agregación de las firmas del comité de sincronización y recorrer un árbol de Merkle-Patricia dentro de un contrato inteligente desplegado en otra cadena implica un volumen considerable de operaciones criptográficas, fundamentalmente sobre curvas elípticas, cuyo costo en gas resulta, en la práctica, prohibitivo. Esta limitación conduce a que los sistemas de interoperabilidad existentes adopten un compromiso distinto, en el cual la verificación criptográfica del estado de la cadena de origen es sustituida por la delegación de dicha verificación en un comité de validadores externos sobre el cual se deposita confianza. Este diseño traslada el problema de seguridad del propio protocolo hacia el comportamiento honesto de un conjunto reducido de actores, y esta delegación de confianza ha constituido, de manera sistemática, el punto de falla explotado en los incidentes de seguridad más costosos sufridos por puentes entre cadenas hasta la fecha [2]. Persiste, en consecuencia, una tensión no resuelta entre dos objetivos igualmente necesarios: por un lado, alcanzar garantías criptográficas equivalentes a las del consenso de la cadena de origen y, por otro, mantener un costo de verificación *on-chain*, tanto en gas como en tiempo de generación de la prueba, que resulte viable para un sistema de uso real.

1.3. Justificación

El ecosistema de herramientas orientadas al desarrollo de circuitos de conocimiento cero capaces de abordar este volumen de cómputo criptográfico se encuentra, a su vez, en un proceso activo de maduración, y ofrece actualmente dos paradigmas de diseño claramente diferenciados. El primero está representado por los lenguajes de dominio específico, orientados exclusivamente a la escritura de circuitos, entre los cuales se encuentra Noir [3], que expone al desarrollador primitivas y un modelo de programación concebidos desde su diseño para la generación eficiente de pruebas. El segundo paradigma corresponde a las máquinas virtuales de conocimiento cero de propósito general, cuyo exponente más reciente es SP1 [4], las cuales permiten probar la ejecución de programas escritos en un lenguaje de propósito general, en este caso Rust, compilados a un conjunto de instrucciones que la máquina virtual ejecuta y prueba sin que el desarrollador deba razonar directamente en términos de restricciones aritméticas. Si bien ambos paradigmas

persiguen resolver el mismo tipo de problema, lo hacen mediante compromisos de diseño distintos entre la expresividad del lenguaje, la facilidad de desarrollo y depuración, y el rendimiento final del circuito resultante. Dado que ambas herramientas son de adopción reciente y se encuentran en desarrollo activo, no existe todavía evidencia empírica consolidada que permita fundamentar la elección de uno u otro paradigma para la implementación de un caso de uso concreto y computacionalmente exigente, como lo es la verificación de consenso e inclusión de transacciones en Ethereum. El presente trabajo se propone generar dicha evidencia mediante la implementación de un mismo problema en ambas tecnologías, bajo condiciones comparables, y la medición sistemática de su comportamiento. Ambas implementaciones se integran, además, dentro del mismo puente *lock-and-mint*, de manera que la comparación entre Noir y SP1 no se limita a un ejercicio aislado de generación de pruebas, sino que se valida en el contexto de una transferencia real de activos digitales entre cadenas.

1.4. Objetivos

1.4.1. Objetivo general

Diseñar, implementar y comparar empíricamente la verificación de consenso de Ethereum y de inclusión de transacciones mediante Noir y SP1, integrando ambas implementaciones en un puente *lock-and-mint* de confianza reducida, con el propósito de determinar cuál de ambas tecnologías ofrece un mejor rendimiento, un menor costo de verificación y una mayor viabilidad práctica para la transferencia segura de activos digitales entre cadenas.

1.4.2. Objetivos específicos

- Investigar los mecanismos de prueba y las arquitecturas de circuitos que subyacen a Noir, en tanto lenguaje de dominio específico, y a SP1, en tanto máquina virtual de conocimiento cero de propósito general basada en Rust.
- Implementar, en ambas tecnologías, la lógica de verificación de encabezados de bloque de Ethereum, de la firma del comité de sincronización y de las pruebas de inclusión de transacciones.
- Medir y comparar las métricas de rendimiento relevantes entre ambas implementaciones, entre las que se incluyen el tiempo de generación de la prueba, el uso de recursos computacionales y de memoria, el costo de verificación *on-chain* en gas, y la complejidad relativa del proceso de desarrollo.

1.5. Hipótesis

El presente trabajo se orienta por la siguiente pregunta de investigación: ¿presenta el paradigma de máquina virtual de conocimiento cero de propósito general, representado por SP1, ventajas significativas en términos de flexibilidad de desarrollo y de tiempos de ejecución frente a un lenguaje de conocimiento cero de dominio específico como Noir, al implementar la verificación de consenso e inclusión de transacciones en Ethereum?

1.6. Delimitación

El presente trabajo se circunscribe a la arquitectura de consenso de Ethereum posterior a *The Merge*, particularmente al mecanismo del comité de sincronización empleado por los clientes ligeros para certificar la finalidad de un bloque, así como a las pruebas de inclusión sobre su árbol de Merkle-Patricia. Las implementaciones desarrolladas y las comparativas de rendimiento realizadas se validan sobre redes de prueba, empleando Ethereum Sepolia como cadena de origen y Optimism Sepolia como cadena de destino, bajo una infraestructura de cómputo estandarizada que permite que las mediciones obtenidas para Noir y para SP1 resulten comparables entre sí. Quedan fuera del alcance de este trabajo tanto la implementación sobre redes principales como la verificación de mecanismos de consenso correspondientes a cadenas distintas de Ethereum.

CAPÍTULO 2

BLOCKCHAIN

2.1. Arquitectura de blockchain

Una *blockchain* es un registro distribuido (*distributed ledger*) de transacciones que se mantiene de forma replicada entre los nodos de una red *peer-to-peer*, sin depender de una autoridad central que valide o custodie su contenido. Su propiedad fundamental es que, una vez que un dato se incorpora al registro, modificarlo o eliminarlo requiere rehacer todo el trabajo criptográfico posterior a ese punto, lo cual se vuelve computacionalmente inviable a medida que la cadena crece. Esta sección describe los cuatro componentes que en conjunto sostienen esa propiedad: la estructura de bloques encadenados por hash, los árboles de Merkle que resumen su contenido, los mecanismos de consenso que deciden qué bloque es válido, y los roles que los distintos tipos de nodos cumplen dentro de la red. Estos conceptos son la base sobre la que se apoyan tanto el diseño de Ethereum (Sección 2.2) como el esquema de verificación de conocimiento cero desarrollado en este trabajo.

2.1.1. Estructura de bloques y encadenamiento por hash

Definición 2.1. Un **bloque** es una estructura de datos compuesta por un *encabezado* (*header*) y un cuerpo. El encabezado contiene, como mínimo, el hash criptográfico del encabezado del bloque anterior, una marca de tiempo, un valor que resume el conjunto de transacciones del bloque (la raíz de un árbol de Merkle, ver Definición 2.2), y metadatos propios del mecanismo de consenso utilizado (por ejemplo, un *nonce* en *Proof-of-Work*). El cuerpo contiene la lista de transacciones incluidas en el bloque.

La **blockchain** propiamente dicha es la secuencia de bloques $B_0, B_1, \ldots, B_n$ donde B_0 es el *bloque génesis* y cada bloque B_i, con $i > 0$, referencia a su predecesor incluyendo $H(B_{i-1})$

en su propio encabezado, siendo H una función de hash criptográfica (típicamente SHA-256 o Keccak-256). Esta construcción encadenada es lo que le da su nombre a la estructura y es la responsable directa de su inmutabilidad: como el hash de un bloque depende del hash del bloque anterior, alterar el contenido de B_i cambia su hash $H(B_i)$, lo que invalida la referencia almacenada en B_{i+1}, y por extensión la de todos los bloques posteriores hasta la cabeza de la cadena. Un atacante que quisiera modificar una transacción confirmada tendría entonces que recalcular el trabajo de consenso de todos los bloques desde ese punto en adelante, y hacerlo más rápido que el resto de la red honesta le suma nuevos bloques a la cadena legítima.

2.1.2. Árboles de Merkle y pruebas de inclusión

Incluir el hash de cada transacción individual en el encabezado del bloque sería costoso en espacio y haría que verificar la pertenencia de una transacción específica implicara descargar el bloque completo. Los árboles de Merkle [5] resuelven ambos problemas.

Definición 2.2. Un **árbol de Merkle** es un árbol binario en el que cada hoja contiene el hash de un elemento de datos (en este contexto, una transacción) y cada nodo interno contiene el hash de la concatenación de sus dos hijos: $h_{padre} = H(h_{izq} \| h_{der})$. El hash almacenado en la raíz, llamado **raíz de Merkle**, es un compromiso criptográfico sobre el conjunto completo de hojas: cambiar cualquier transacción altera su hash hoja y, por propagación, la raíz.

Esta estructura permite generar una **prueba de inclusión** (*Merkle proof*) de que una transacción t forma parte de un bloque sin necesitar el bloque completo: basta con entregar t junto con los hashes hermanos de cada nivel del camino desde su hoja hasta la raíz. Quien verifica recalcula los hashes de ese camino y comprueba que el resultado coincide con la raíz de Merkle publicada en el encabezado del bloque. Para un árbol con n hojas, la prueba tiene tamaño $O(\log n)$, en contraste con el $O(n)$ que implicaría transmitir todas las transacciones. Ethereum extiende esta idea con *Merkle-Patricia tries*, que además de las transacciones permiten probar el estado de una cuenta o el resultado de una ejecución (Sección 2.2); esta capacidad de generar pruebas de inclusión compactas y verificables es, precisamente, el mecanismo que este trabajo utiliza para certificar *on-chain*, mediante una prueba de conocimiento cero, que una transacción de bloqueo fue efectivamente incluida en la cadena de origen.

2.1.3. Mecanismos de consenso

Que cada nodo mantenga una copia del registro no es suficiente: en una red sin autoridad central, distintos nodos pueden recibir transacciones en distinto orden o proponer bloques en simultáneo, y es necesario que todos converjan a una única secuencia de bloques válida. Este es un caso particular del problema de los generales bizantinos [6]: lograr acuerdo entre participantes distribuidos cuando una fracción de ellos puede ser maliciosa o fallar arbitrariamente. Un **mecanismo de consenso** es el protocolo que resuelve este problema definiendo, ante bloques o cadenas en conflicto, cuál es la versión canónica.

Los dos mecanismos dominantes son:

- **Proof-of-Work (PoW)**: introducido por Bitcoin [7], exige que quien propone un bloque resuelva un problema computacional costoso (encontrar un *nonce* tal que $H(\text{header})$ sea menor a un umbral de dificultad). La cadena válida es la que acumula mayor trabajo computacional total, lo que hace que reescribir el historial requiera superar la capacidad de cómputo de toda la red honesta. Su finalidad es *probabilística*: la confianza en que un bloque no será revertido crece con la cantidad de bloques confirmados sobre él, pero nunca es absoluta.

- **Proof-of-Stake (PoS)**: sustituye el costo computacional por un costo económico. Los participantes (*validadores*) bloquean una cantidad de la criptomoneda nativa como garantía y son seleccionados para proponer y atestiguar bloques en proporción a su participación bloqueada (*stake*). Un comportamiento malicioso detectable (como firmar dos bloques en conflicto para la misma posición) resulta en la pérdida de parte o la totalidad de esa garantía (*slashing*). A diferencia de PoW, las variantes modernas de PoS —como la que adoptó Ethereum tras *The Merge*— pueden alcanzar **finalidad determinística**: una vez que un bloque acumula suficientes atestiguaciones, revertirlo es no solo improbable sino que exige destruir económicamente a los validadores responsables. Este mecanismo de finalidad, y el comité que la certifica en Ethereum, se retoma en la Sección 2.2 y es la raíz de confianza sobre la que se construye el puente propuesto en este trabajo.

2.1.4. Roles de nodos y topología de red

No todos los participantes de una red blockchain cumplen la misma función ni requieren los mismos recursos:

- **Nodos completos** (*full nodes*): descargan y validan de forma independiente cada bloque y cada transacción desde el bloque génesis, manteniendo una copia íntegra del estado de la red. No confían en ningún tercero: verifican por sí mismos que se respetaron las reglas de consenso y de validez de transacciones.

- **Mineros o validadores**: además de validar, participan activamente en la producción de nuevos bloques (resolviendo el rompecabezas de PoW o siendo seleccionados y atestiguando en PoS).

- **Clientes ligeros** (*light clients*): no almacenan la cadena completa ni reejecutan todas las transacciones; en su lugar, confían en los encabezados de bloque —en particular en su raíz de Merkle y, en redes PoS, en la firma del comité que certifica su finalidad— para verificar de forma eficiente si una transacción específica fue incluida, mediante una prueba de inclusión como la descrita en la Sección 2.1.2. Este es precisamente el modelo

de confianza que adopta el *relayer off-chain* del puente diseñado en este trabajo: no re-ejecuta la cadena de origen, sino que verifica una prueba de inclusión respaldada por la firma del comité de sincronización de Ethereum.

Todos estos roles se comunican mediante una red *peer-to-peer* de propagación por *gossip*, en la que cada nodo retransmite a sus pares los bloques y transacciones que recibe y considera válidos, sin que exista un servidor central que coordine la difusión de la información.

2.2. Ethereum (Estructura de bloques)

2.3. Aztec (Estructura de bloques)

2.4. Puentes

CAPÍTULO 3

CONOCIMIENTO CERO

3.1. Fundamentos

El capítulo anterior describió los mecanismos mediante los cuales una red blockchain garantiza la integridad de su registro, desde el encadenamiento de bloques por hash hasta las pruebas de inclusión sobre árboles de Merkle y el modelo de confianza de los clientes ligeros (Sección 2.1). En todos esos casos, quien verifica una afirmación sobre la cadena necesita disponer de los datos pertinentes y ejecutar por sí mismo el cómputo correspondiente. Las pruebas de conocimiento cero ofrecen una alternativa a este modelo, puesto que permiten convencer a un verificador de que una afirmación es verdadera, o de que una computación se ejecutó correctamente, sin que este deba repetir dicha computación.

La presente sección no pretende ofrecer un tratamiento exhaustivo de la teoría de las pruebas de conocimiento cero, sino establecer el vocabulario mínimo necesario para comprender las herramientas analizadas en el resto del capítulo. Para un desarrollo completo de los aspectos teóricos se remite al lector a la obra de Thaler [8], cuya terminología se adopta en lo que sigue.

3.1.1. Concepto de conocimiento cero

En su acepción habitual, demostrar una afirmación consiste en presentar evidencia que permita a otra parte comprobar su veracidad. Si alguien afirma conocer un valor w cuyo hash criptográfico coincide con un valor público y, es decir, un valor tal que $H(w) = y$, la forma más directa de demostrarlo es entregar w para que el receptor calcule $H(w)$ y lo compare con y. Esta demostración resulta convincente, pero el receptor termina conociendo el valor que se pretendía mantener en secreto. Goldwasser, Micali y Rackoff demostraron que es posible convencer a otra parte de una afirmación de este tipo sin transferirle ningún conocimiento adicional a la validez de la afirmación misma, y denominaron a este tipo de demostraciones

pruebas de conocimiento cero [9]. Posteriormente, Goldreich, Micali y Wigderson demostraron que, bajo supuestos criptográficos estándar, toda afirmación cuya veracidad puede comprobarse eficientemente a partir de un certificado admite una prueba de este tipo [10].

El ejemplo anterior permite introducir los cinco términos que se emplean en el resto del capítulo.

- El **probador** (*prover*), denotado $\mathcal{P}$, es la parte que afirma la validez de un enunciado y genera la evidencia que la respalda.
- El **verificador** (*verifier*), denotado $\mathcal{V}$, es la parte que examina dicha evidencia y decide si la acepta o la rechaza. Se asume que el verificador dispone de recursos de cómputo limitados, por lo que su trabajo debe ser eficiente.
- El **enunciado** (*statement*), denotado x, es la afirmación que se desea demostrar, expresada a partir de datos públicos conocidos por ambas partes. En el ejemplo, el enunciado es que existe un valor cuyo hash es y.
- El **testigo** (*witness*), denotado w, es la información que el probador utiliza para demostrar el enunciado y que no necesita entregar al verificador. En el ejemplo, el testigo es la preimagen w.
- La **prueba** (*proof*), denotada π, es el objeto que el probador genera a partir del enunciado y del testigo, y que el verificador examina para tomar su decisión.

Una **relación** $\mathcal{R}$ es un conjunto de pares (x, w) cuya pertenencia puede comprobarse eficientemente, y un enunciado x es verdadero cuando existe algún testigo w tal que $(x, w) \in \mathcal{R}$ [11]. En el ejemplo, la relación está formada por los pares (y, w) que satisfacen $H(w) = y$. Las herramientas que se analizan en las secciones siguientes reflejan esta distinción mediante las nociones de *entradas públicas*, que forman parte del enunciado, y *entradas privadas*, que forman parte del testigo.

Conviene aclarar que el conocimiento cero y la privacidad no son necesariamente lo mismo en las aplicaciones blockchain. Algunas aplicaciones emplean estas pruebas para ocultar información, como Zerocash, que valida pagos sin revelar su origen, su destino ni su monto [12]. Sin embargo, los sistemas de prueba modernos ofrecen además una propiedad independiente del conocimiento cero, que consiste en la posibilidad de verificar una computación en un tiempo muy inferior al que requiere ejecutarla. Walfish y Blumberg describen este objetivo como la verificación de computaciones sin reejecutarlas, de manera que una parte con recursos limitados puede delegar un cómputo en otra parte no confiable y comprobar después, a bajo costo, que el resultado recibido es correcto [13]. En muchas aplicaciones blockchain, los datos involucrados son públicos y el valor de la prueba reside precisamente en esta segunda propiedad. La literatura y la industria emplean de todos modos la expresión *prueba de conocimiento cero* para referirse a estos sistemas, convención que se adopta también en el presente trabajo.

Un ejemplo relacionado con el Capítulo 2 permite apreciar esta diferencia. Una prueba de inclusión en un árbol de Merkle (Sección 2.1.2) puede formularse como un enunciado, cuyo contenido es que un elemento t pertenece al árbol de raíz r, y un testigo, formado por los hashes hermanos del camino que une a t con r. Ninguno de estos datos es secreto, ya que todos pueden obtenerse de la cadena. El interés de expresar esta verificación como un enunciado reside en la posibilidad de combinarla con otras condiciones de validez, como la comprobación de que la raíz r pertenece a un encabezado certificado por los validadores de la cadena, y de demostrar el conjunto mediante una única prueba cuya verificación resulte económica.

3.1.2. Propiedades fundamentales

Un sistema de prueba de conocimiento cero se caracteriza por tres propiedades, cada una de las cuales protege a una de las partes frente a un comportamiento incorrecto de la otra [9, 8].

La **completitud** (*completeness*) garantiza que, si el enunciado es verdadero y el probador sigue el protocolo utilizando un testigo válido, el verificador acepta la prueba. Esta propiedad protege al probador honesto, puesto que asegura que una demostración correcta no será rechazada.

La **solidez** (*soundness*) garantiza que, si el enunciado es falso, ningún probador logra que el verificador acepte, salvo con una probabilidad *despreciable*, es decir, tan pequeña que en la práctica puede considerarse nula. Esta propiedad protege al verificador frente a un probador deshonesto, denotado $\mathcal{P}^*$, que intenta hacer pasar por verdadera una afirmación falsa. Cuando esta garantía depende de la dificultad de algún problema computacional, como ocurre en todos los sistemas analizados en este capítulo, el sistema se denomina técnicamente *argumento* en lugar de *prueba* [14].

En ciertos casos la solidez resulta insuficiente. En el ejemplo de la preimagen, prácticamente todo valor y posee alguna preimagen, de manera que demostrar su existencia carece de interés. Lo que se pretende demostrar es que el probador *conoce* una. Esta exigencia se denomina **solidez de conocimiento** (*knowledge soundness*) y establece que de todo probador capaz de convencer al verificador podría obtenerse un testigo válido, lo que implica que dicho probador efectivamente lo conocía [15]. Los sistemas que cumplen esta propiedad se denominan *argumentos de conocimiento*.

El **conocimiento cero** (*zero-knowledge*) garantiza que el verificador no obtiene de la prueba ninguna información sobre el testigo más allá de la validez del enunciado. Esta propiedad protege al probador, y su formalización se basa en mostrar que todo lo que el verificador observa podría haberse generado a partir del enunciado sin conocer el testigo [9].

La importancia relativa de cada propiedad depende de la aplicación. En un puente entre cadenas, el verificador es un contrato inteligente que acuña activos cuando acepta una prueba, y el adversario principal es un probador que intenta obtener esa acuñación a partir de un evento que nunca ocurrió en la cadena de origen. En este escenario la propiedad crítica es la solidez,

puesto que su violación permitiría acuñar activos sin respaldo. El conocimiento cero, en cambio, resulta secundario cuando los datos que se prueban son públicos.

3.1.3. Pruebas interactivas y no interactivas

En el modelo original de Goldwasser, Micali y Rackoff, probador y verificador intercambian varios mensajes antes de que el verificador tome su decisión [9]. El verificador envía *desafíos* elegidos al azar, y la solidez se apoya en que el probador no puede anticiparlos. Este tipo de protocolos se denomina **interactivo** y presenta limitaciones importantes para su uso en una red blockchain. La interacción exige que ambas partes estén disponibles al mismo tiempo, debe repetirse con cada verificador y solo convence a quien eligió los desafíos. Un contrato inteligente, además, se ejecuta de manera determinista como respuesta a una transacción y no puede mantener un diálogo de varias rondas con el probador.

Una prueba **no interactiva** consiste, en cambio, en un único mensaje π que el probador genera por sí solo y que cualquier parte puede verificar en cualquier momento. Blum, Feldman y Micali demostraron que este tipo de pruebas es posible cuando ambas partes comparten ciertos parámetros públicos generados de antemano mediante un procedimiento denominado **configuración inicial** (*setup*) [16]. Si ese procedimiento emplea valores secretos que deben destruirse, porque su conocimiento permitiría generar pruebas falsas, se habla de una **configuración confiable** (*trusted setup*). Si solo emplea aleatoriedad pública, la configuración se denomina **transparente**. Esta distinción es una de las diferencias principales entre las familias de sistemas presentadas en la Sección 3.1.5.

La técnica más difundida para eliminar la interacción es la **transformación de Fiat-Shamir** [17], que reemplaza cada desafío aleatorio del verificador por el hash de los mensajes enviados previamente por el probador. Dado que el probador no puede predecir la salida de la función de hash antes de fijar sus mensajes, el desafío resulta para él tan impredecible como si lo hubiera elegido el verificador, y la prueba puede generarse sin intervención de este último. Una prueba así obtenida puede incluirse en una transacción y ser verificada tanto por un contrato inteligente como por cualquier nodo que valide el bloque, lo que la convierte en un objeto persistente y verificable públicamente.

3.1.4. De una computación a una prueba

Esta subsección describe en primer lugar el enfoque basado en circuitos aritméticos y restricciones, que constituye el fundamento de herramientas como Circom y Noir. En este enfoque, la computación que se desea probar no se expresa como un programa escrito en un lenguaje de programación, sino mediante una representación matemática cuya construcción se denomina **aritmetización** [8]. Todos los valores que intervienen en esa representación son elementos de un **cuerpo finito** $\mathbb{F}_p$, formado por los enteros $\{0, 1, \ldots, p-1\}$ con la suma y el producto calculados módulo un número primo p, de manera que, a diferencia de lo que ocurre en un procesador convencional, todo resultado se reduce módulo p. Esta descripción no se aplica de

igual modo a todos los sistemas de prueba. Las zkVM, presentadas al final de esta subsección, no requieren que el desarrollador exprese la computación de esta forma y emplean internamente representaciones distintas, que se analizan en la Sección 3.4.

Circuitos y restricciones. Un **circuito aritmético** representa la computación como una red de compuertas de suma y de producto sobre $\mathbb{F}_p$. Sin embargo, un sistema de prueba no necesita repetir la computación, sino comprobar que un conjunto de valores propuesto por el probador es coherente con ella. Por ese motivo, el circuito se expresa como un **sistema de restricciones**, es decir, un conjunto de ecuaciones sobre $\mathbb{F}_p$ que deben cumplirse simultáneamente y cuyas incógnitas incluyen las entradas, las salidas y todos los valores intermedios. Para demostrar que se conoce un valor w tal que $w^3 + w + 5 = 35$, por ejemplo, la computación puede descomponerse en tres restricciones con dos variables intermedias v_1 y v_2.

$$w \cdot w = v_1, \qquad v_1 \cdot w = v_2, \qquad v_2 + w + 5 = 35. \tag{3.1}$$

El testigo que satisface este sistema no es solamente el valor $w = 3$, sino la asignación completa $(w, v_1, v_2) = (3, 9, 27)$. En general, el testigo de un circuito está formado por todos los valores que surgen al ejecutar la computación. El formato de restricciones más difundido es el sistema de restricciones de rango uno (*Rank-1 Constraint System*, R1CS) [18], que se retoma en la Sección 3.2.

Esta forma de representación tiene dos consecuencias prácticas. La primera es que una restricción omitida no produce un error de ejecución, sino que amplía el conjunto de testigos aceptados, lo que permite a un probador deshonesto satisfacer el sistema con valores que no corresponden a la computación prevista. Los circuitos insuficientemente restringidos constituyen, de hecho, una categoría recurrente entre las 141 vulnerabilidades reales analizadas por Chaliasos et al. [19]. La segunda consecuencia es que las operaciones que no son nativas del cuerpo finito resultan costosas. Las funciones de hash SHA-256 y Keccak-256, empleadas en el consenso y en las estructuras de datos de Ethereum, operan sobre bits, y cada bit debe representarse como un elemento de $\mathbb{F}_p$ restringido a los valores 0 y 1, por lo que una única evaluación de estas funciones requiere un número de restricciones muy superior al de una operación aritmética nativa. Los sistemas actuales mitigan este costo mediante tablas de búsqueda (*lookup tables*), técnica que se retoma en la Sección 3.3.

Generación y verificación de la prueba. Una vez definida la representación, el proceso completo se desarrolla en cuatro etapas, que la Figura 3.1 representa de manera esquemática.

1. **Configuración inicial.** A partir del sistema de restricciones se generan los parámetros públicos que emplean el probador y el verificador. Esta etapa se realiza una única vez y, en los sistemas transparentes, no requiere valores secretos.

2. **Generación del testigo.** El probador ejecuta la computación sobre las entradas concretas y registra todos los valores que intervienen en ella.

3. **Generación de la prueba.** El probador transforma el testigo en una prueba π mediante operaciones criptográficas. Esta es, con diferencia, la etapa más costosa del proceso, tanto en tiempo como en memoria.

4. **Verificación.** El verificador comprueba la prueba π a partir del enunciado x y de los parámetros públicos, sin acceso al testigo ni a la computación original, y la acepta o la rechaza.

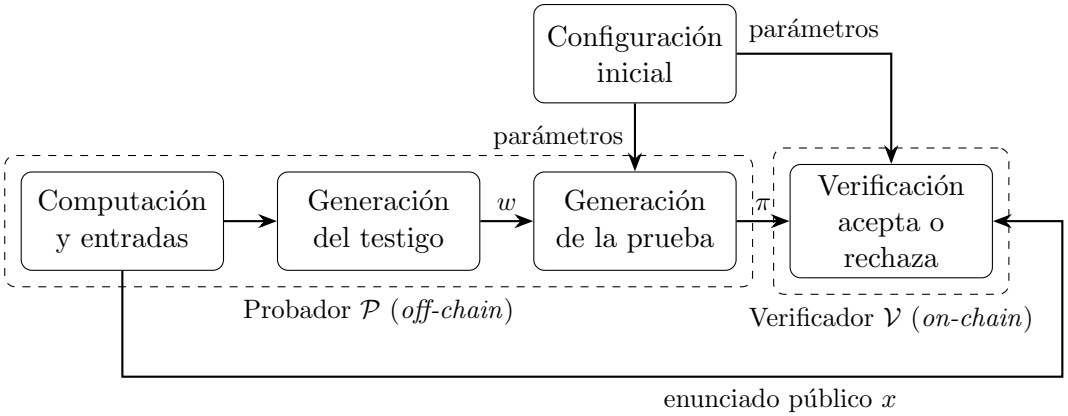


Figura 3.1: Etapas de generación y verificación de una prueba.

El reparto del trabajo entre ambas partes es marcadamente asimétrico. El costo de generar la prueba crece al menos de manera lineal con el tamaño de la computación y supera ampliamente el de ejecutarla, mientras que el costo de verificarla es, en los sistemas que se describen en la Sección 3.1.5, constante o muy pequeño en comparación. Por ese motivo, en las aplicaciones blockchain la prueba se genera fuera de la cadena (*off-chain*), en equipos con recursos suficientes, y se verifica dentro de la cadena (*on-chain*), en un contrato inteligente que recibe la prueba y el enunciado como datos de una transacción. Conviene subrayar, además, que la garantía obtenida se limita a que el probador conoce un testigo que satisface las restricciones para el enunciado dado. Su valor depende, por lo tanto, de que las restricciones expresen fielmente la afirmación que se pretende establecer, y alcanza únicamente a los valores que forman parte del enunciado, de modo que todo dato que el verificador utilice para tomar su decisión, como el destinatario o el monto de una transferencia, debe ser una entrada pública o estar vinculado criptográficamente a una.

Del circuito a la ejecución de programas. En el enfoque descrito, cada computación da lugar a un sistema de restricciones específico, cuyo tamaño queda fijado de antemano. Los ciclos deben desplegarse con una cantidad máxima de iteraciones conocida al construir el circuito, las bifurcaciones que dependen de los datos obligan a representar ambas ramas, y toda modificación de la lógica produce un circuito distinto. Circom y Noir, analizados en las Secciones 3.2 y 3.3, pertenecen a este enfoque, aunque difieren en el nivel de abstracción que ofrecen al desarrollador.

Existe una forma conceptualmente distinta de plantear el problema. En lugar de demostrar que un testigo satisface las restricciones de una computación particular, puede demostrarse que

un programa arbitrario, ejecutado sobre un procesador determinado, produjo cierto resultado. En este caso las restricciones no describen el programa sino el procesador, puesto que expresan que cada paso de la ejecución es una transición válida de acuerdo con la instrucción correspondiente. El testigo pasa a ser la **traza de ejecución**, es decir, el registro del estado del procesador en cada paso, y el programa se convierte en un dato más del enunciado, de manera que un único sistema de restricciones permite probar la ejecución de cualquier programa. Ben-Sasson et al. materializaron esta idea al construir pruebas sucintas para la ejecución de programas escritos en el lenguaje C [20].

Una **máquina virtual de conocimiento cero** (*zero-knowledge virtual machine*, zkVM) aplica este principio a un conjunto de instrucciones concreto. La zkVM ejecuta un programa escrito en un lenguaje de propósito general y compilado con herramientas convencionales, registra su traza y genera una prueba de que la ejecución fue correcta y produjo determinadas salidas públicas. SP1, analizado en la Sección 3.4, ejecuta de este modo programas escritos en Rust y compilados al conjunto de instrucciones RISC-V [4]. La diferencia entre ambos enfoques no reside en lo que pueden demostrar, sino en los compromisos que adoptan. En el enfoque basado en circuitos, el desarrollador razona en términos de restricciones, lo que exige un conocimiento especializado pero permite ajustar la representación a la computación. En el enfoque basado en una zkVM, el desarrollador razona en términos de un programa convencional, lo que facilita el desarrollo, la reutilización de bibliotecas y la depuración, a cambio de un costo adicional por cada instrucción ejecutada. Este compromiso es el que motiva la comparación entre Noir y SP1 planteada en la hipótesis del presente trabajo.

3.1.5. SNARKs y STARKs

Las propiedades descritas en la Sección 3.1.2 no se refieren al tamaño de la prueba ni al costo de verificarla, aspectos determinantes en las aplicaciones blockchain, ya que la prueba debe transmitirse en una transacción y su verificación se paga en gas. Un sistema de prueba es **sucinto** cuando tanto el tamaño de la prueba como el tiempo de verificación son muy pequeños en comparación con el tamaño de la computación probada [8]. Las dos familias de sistemas sucintos más utilizadas en la práctica son los SNARKs y los STARKs.

Ambas familias comparten una idea común, que consiste en codificar la computación como relaciones entre polinomios y comprobarlas en unos pocos puntos elegidos al azar, lo que se fundamenta en el principio de Schwartz–Zippel [21, 22] y reduce drásticamente el trabajo del verificador. Se diferencian principalmente en el mecanismo con el que el probador se compromete con esos polinomios, basado en curvas elípticas en los SNARKs más difundidos, como ocurre con el esquema KZG [23], y en funciones de hash en los STARKs, mediante árboles de Merkle y el protocolo FRI [24].

SNARKs. La sigla SNARK (*Succinct Non-interactive ARgument of Knowledge*) describe un argumento sucinto, no interactivo y con solidez de conocimiento [25]. Un SNARK que además

es de conocimiento cero se denomina zk-SNARK. Las construcciones más difundidas se basan en curvas elípticas y emplean una operación criptográfica denominada *emparejamiento bilineal*, definida entre grupos asociados a dichas curvas. El sistema de Groth, conocido como Groth16, produce pruebas del orden de pocos cientos de bytes que se verifican con un número constante de operaciones, independientemente del tamaño de la computación [26]. PLONK, otro sistema de esta familia, admite una configuración universal, válida para cualquier circuito hasta un tamaño máximo, en lugar de la configuración específica para cada circuito que requiere Groth16 [27]. Noir emplea por defecto UltraHonk, un sistema derivado de PLONK que se describe en la Sección 3.3.

La principal limitación de estos sistemas es que requieren una configuración confiable, cuyos valores secretos permitirían generar pruebas falsas si no se destruyen. Para reducir esta confianza se realizan ceremonias con numerosos participantes, cuyo resultado es seguro siempre que al menos uno de ellos haya destruido su contribución [28]. En contrapartida, estos sistemas ofrecen la verificación más económica disponible en Ethereum, que desde 2017 incorpora contratos precompilados para las operaciones sobre la curva elíptica `alt_bn128`, también denominada BN254 [29, 30].

STARKs. La sigla STARK (*Scalable Transparent ARgument of Knowledge*) designa una familia de sistemas propuesta por Ben-Sasson et al. que prescinde de la configuración confiable [31]. En lugar de curvas elípticas, los STARKs se basan únicamente en funciones de hash resistentes a colisiones, lo que los hace además plausiblemente resistentes a ataques mediante computación cuántica. Son *transparentes*, en el sentido descrito en la Sección 3.1.3, y *escalables*, ya que el tiempo de generación de la prueba crece de manera casi lineal con el tamaño de la computación, lo que resulta conveniente para computaciones extensas como la ejecución de un programa completo. Su principal desventaja es el tamaño de la prueba, varios órdenes de magnitud mayor que el de una prueba Groth16, lo que encarece considerablemente su verificación directa en Ethereum. La Tabla 3.1 resume las diferencias entre ambas familias.

Tabla 3.1: Diferencias conceptuales entre SNARKs y STARKs.

Característica	**SNARK**	**STARK**
Tamaño de la prueba	Muy pequeño	Mayor
Verificación	Muy eficiente	Eficiente, pero generalmente más costosa
Configuración inicial	Algunas construcciones requieren una configuración confiable	Transparente
Base criptográfica	Habitualmente curvas elípticas y emparejamientos	Funciones de hash y FRI
Contexto de uso	Circuitos y verificación compacta	Computaciones extensas y sistemas de tipo zkVM

Ambas familias no son excluyentes. Es posible verificar una prueba STARK dentro de un circuito y generar sobre esa verificación una prueba SNARK, de manera que el resultado combine la escalabilidad del probador de los STARKs con la verificación económica de los SNARKs. Esta composición es la que emplean las zkVM modernas para que sus pruebas puedan verificarse en Ethereum, y su aplicación en SP1 se analiza en la Sección 3.4.

Relación con los puentes entre cadenas. Los conceptos presentados permiten reformular el problema de los puentes descrito en la Sección 2.4. Un contrato inteligente desplegado en la cadena de destino necesita establecer la validez de una afirmación sobre otra cadena, por ejemplo, que una transacción de bloqueo fue incluida en un bloque certificado por los validadores de la cadena de origen. Un cliente ligero (Sección 2.1.4) establecería esta afirmación verificando la firma que certifica el encabezado y la prueba de inclusión de la transacción, pero reproducir esas verificaciones dentro de un contrato inteligente resulta demasiado costoso. Si ambas verificaciones se expresan como un único enunciado, el trabajo costoso queda a cargo de un probador fuera de la cadena, y el contrato solo necesita verificar una prueba sucinta. La solidez del sistema garantiza entonces que ningún probador puede obtener la aceptación del contrato a partir de un evento inexistente, de modo que la seguridad del puente deja de depender de la honestidad de un intermediario. Xie et al. demostraron la viabilidad práctica de este enfoque [32]. El diseño concreto de esta verificación para el puente desarrollado en el presente trabajo se expone en el Capítulo 4.

3.2. Circom

3.3. Noir

3.4. SP1

3.5. Otros (Risc0, Succinct, Aligned, Cairo, Halo2, Plonky3)

CAPÍTULO 4

ZKBRIDGE

4.1. Implementación de un puente Lock-and-Mint

4.2. zkBridge de Berkeley RDI

4.3. Diseño e Implementación en Noir

4.4. Benchmarks

CAPÍTULO 5

CONCLUSIÓN

REFERENCIAS

[1] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. 2014. URL: https://ethereum.org/en/whitepaper/.

[2] Mengya Zhang, Xiaokuan Zhang, Josh Barbee, Yinqian Zhang y Zhiqiang Lin. *SoK: Security of Cross-chain Bridges: Attack Surfaces, Defenses, and Open Problems*. arXiv:2312.12573 [cs.CR]. 2023. URL: https://arxiv.org/abs/2312.12573.

[3] Aztec Labs. *Noir: A Domain Specific Language for Zero-Knowledge Proofs*. 2024. URL: https://noir-lang.org/docs/.

[4] Succinct Labs. *SP1: A Zero-Knowledge Virtual Machine for Rust*. 2024. URL: https://docs.succinct.xyz/docs/sp1/introduction.

[5] Ralph C. Merkle. «A Digital Signature Based on a Conventional Encryption Function». En: *Advances in Cryptology — CRYPTO '87* (1988), págs. 369-378. DOI: 10.1007/3-540-48184-2_32.

[6] Leslie Lamport, Robert Shostak y Marshall Pease. «The Byzantine Generals Problem». En: *ACM Transactions on Programming Languages and Systems* 4.3 (1982), págs. 382-401. URL: https://lamport.azurewebsites.net/pubs/byz.pdf.

[7] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. URL: https://bitcoin.org/bitcoin.pdf.

[8] Justin Thaler. «Proofs, Arguments, and Zero-Knowledge». En: *Foundations and Trends in Privacy and Security* 4.2–4 (2022), págs. 117-660. DOI: 10.1561/3300000030. URL: https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.pdf.

[9] Shafi Goldwasser, Silvio Micali y Charles Rackoff. «The Knowledge Complexity of Interactive Proof Systems». En: *SIAM Journal on Computing* 18.1 (1989), págs. 186-208. DOI: 10.1137/0218012. URL: https://doi.org/10.1137/0218012.

[10] Oded Goldreich, Silvio Micali y Avi Wigderson. «Proofs that Yield Nothing but Their Validity or All Languages in NP Have Zero-Knowledge Proof Systems». En: *Journal of the ACM* 38.3 (1991), págs. 690-728. DOI: 10.1145/116825.116852. URL: https://doi.org/10.1145/116825.116852.

[11] Oded Goldreich. *Foundations of Cryptography. Volume 1: Basic Tools*. Cambridge University Press, 2001. DOI: 10.1017/CBO9780511546891. URL: https://doi.org/10.1017/CBO9780511546891.

[12] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer y Madars Virza. «Zerocash: Decentralized Anonymous Payments from Bitcoin». En: *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, págs. 459-474. DOI: 10.1109/SP.2014.36. URL: https://doi.org/10.1109/SP.2014.36.

[13] Michael Walfish y Andrew J. Blumberg. «Verifying Computations without Reexecuting Them». En: *Communications of the ACM* 58.2 (2015), págs. 74-84. DOI: 10.1145/2641562. URL: https://doi.org/10.1145/2641562.

[14] Gilles Brassard, David Chaum y Claude Crépeau. «Minimum Disclosure Proofs of Knowledge». En: *Journal of Computer and System Sciences* 37.2 (1988), págs. 156-189. DOI: 10.1016/0022-0000(88)90005-0. URL: https://doi.org/10.1016/0022-0000(88)90005-0.

[15] Mihir Bellare y Oded Goldreich. «On Defining Proofs of Knowledge». En: *Advances in Cryptology — CRYPTO '92*. Vol. 740. Lecture Notes in Computer Science. Springer, 1993, págs. 390-420. DOI: 10.1007/3-540-48071-4_28. URL: https://doi.org/10.1007/3-540-48071-4_28.

[16] Manuel Blum, Paul Feldman y Silvio Micali. «Non-Interactive Zero-Knowledge and Its Applications». En: *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing (STOC '88)*. ACM, 1988, págs. 103-112. DOI: 10.1145/62212.62222. URL: https://doi.org/10.1145/62212.62222.

[17] Amos Fiat y Adi Shamir. «How to Prove Yourself: Practical Solutions to Identification and Signature Problems». En: *Advances in Cryptology — CRYPTO '86*. Vol. 263. Lecture Notes in Computer Science. Springer, 1987, págs. 186-194. DOI: 10.1007/3-540-47721-7_12. URL: https://doi.org/10.1007/3-540-47721-7_12.

[18] Bryan Parno, Jon Howell, Craig Gentry y Mariana Raykova. «Pinocchio: Nearly Practical Verifiable Computation». En: *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, págs. 238-252. DOI: 10.1109/SP.2013.47. URL: https://doi.org/10.1109/SP.2013.47.

[19] Stefanos Chaliasos, Jens Ernstberger, David Theodore, David Wong, Mohammad Jahanara y Benjamin Livshits. *SoK: What Don't We Know? Understanding Security Vulnerabilities in SNARKs*. arXiv:2402.15293 [cs.CR]. 2024. URL: https://arxiv.org/abs/2402.15293.

[20] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer y Madars Virza. «SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge». En: *Advances in Cryptology — CRYPTO 2013*. Vol. 8043. Lecture Notes in Computer Science. Springer, 2013, págs. 90-108. DOI: 10.1007/978-3-642-40084-1_6. URL: https://doi.org/10.1007/978-3-642-40084-1_6.

[21] Jacob T. Schwartz. «Fast Probabilistic Algorithms for Verification of Polynomial Identities». En: *Journal of the ACM* 27.4 (1980), págs. 701-717. DOI: 10.1145/322217.322225. URL: https://doi.org/10.1145/322217.322225.

[22] Richard Zippel. «Probabilistic Algorithms for Sparse Polynomials». En: *Symbolic and Algebraic Computation (EUROSAM '79)*. Vol. 72. Lecture Notes in Computer Science. Springer, 1979, págs. 216-226. DOI: 10.1007/3-540-09519-5_73. URL: https://doi.org/10.1007/3-540-09519-5_73.

[23] Aniket Kate, Gregory M. Zaverucha y Ian Goldberg. «Constant-Size Commitments to Polynomials and Their Applications». En: *Advances in Cryptology — ASIACRYPT 2010*. Vol. 6477. Lecture Notes in Computer Science. Springer, 2010, págs. 177-194. DOI: 10.1007/978-3-642-17373-8_11. URL: https://doi.org/10.1007/978-3-642-17373-8_11.

[24] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh y Michael Riabzev. «Fast Reed-Solomon Interactive Oracle Proofs of Proximity». En: *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Vol. 107. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, 14:1-14:17. DOI: 10.4230/LIPIcs.ICALP.2018.14. URL: https://doi.org/10.4230/LIPIcs.ICALP.2018.14.

[25] Nir Bitansky, Ran Canetti, Alessandro Chiesa y Eran Tromer. «From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again». En: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS '12)*. ACM, 2012, págs. 326-349. DOI: 10.1145/2090236.2090263. URL: https://doi.org/10.1145/2090236.2090263.

[26] Jens Groth. «On the Size of Pairing-Based Non-interactive Arguments». En: *Advances in Cryptology — EUROCRYPT 2016*. Vol. 9666. Lecture Notes in Computer Science. Springer, 2016, págs. 305-326. DOI: 10.1007/978-3-662-49896-5_11. URL: https://doi.org/10.1007/978-3-662-49896-5_11.

[27] Ariel Gabizon, Zachary J. Williamson y Oana Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive, Paper 2019/953. 2019. URL: https://eprint.iacr.org/2019/953.

[28] Sean Bowe, Ariel Gabizon y Ian Miers. *Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model*. Cryptology ePrint Archive, Paper 2017/1050. 2017. URL: https://eprint.iacr.org/2017/1050.

[29] Christian Reitwiessner. *EIP-196: Precompiled Contracts for Addition and Scalar Multiplication on the Elliptic Curve alt_bn128*. Ethereum Improvement Proposals. 2017. URL: https://eips.ethereum.org/EIPS/eip-196.

[30] Vitalik Buterin y Christian Reitwiessner. *EIP-197: Precompiled Contracts for Optimal Ate Pairing Check on the Elliptic Curve alt_bn128*. Ethereum Improvement Proposals. 2017. URL: https://eips.ethereum.org/EIPS/eip-197.

[31] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh y Michael Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. Cryptology ePrint Archive, Paper 2018/046. 2018. URL: https://eprint.iacr.org/2018/046.

[32] Tiancheng Xie, Jiaheng Zhang, Zerui Cheng, Fan Zhang, Yupeng Zhang, Yongzheng Jia, Dan Boneh y Dawn Song. «zkBridge: Trustless Cross-chain Bridges Made Practical». En: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. ACM, 2022, págs. 3003-3017. DOI: 10.1145/3548606.3560652. URL: https://doi.org/10.1145/3548606.3560652.

APÉNDICE

A.1. A